

САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет прикладной математики – процессов управления

Кафедра информационных систем

Маслакова Анастасия Андреевна

Выпускная квалификационная работа

**«Веб-платформа интерактивной визуализации и анализа
многомерных численных алгоритмов»**

Уровень образования: бакалавриат

Направление: 01.03.02 Прикладная математика и информатика

Основная образовательная программа СВ.5005.2022: «Прикладная математика,
фундаментальная информатика и программирование»

Научный руководитель

доктор физ.-мат. наук, профессор

Матросов Александр Васильевич

Рецензент

Доцент кафедры моделирования экономических систем,

кандидат физ.-мат. наук

Ковшов Александр Михайлович

Санкт-Петербург

2026

Содержание

Введение	3
Постановка задачи	5
Обзор литературы	6
1 Технология WebAssembly и высокопроизводительные вычисления в браузерной среде	8
1.1 Определение и назначение WebAssembly	8
1.2 Архитектура и модель исполнения	8
1.3 Компиляция нативного кода в WebAssembly	9
1.4 Многопоточность в браузерной среде	11
1.5 Производительность WebAssembly	11
1.6 Спецификации BLAS и LAPACK	12
2 Проектирование архитектуры платформы	14
2.1 Требования к платформе и функциональные возможности	14
2.2 Выбор технологического стека	14
2.3 Архитектура прототипа	15
2.4 Проектирование структур данных и интерфейсов взаимодействия	17
3 Реализация прототипа	22
3.1 Файловая структура проекта	22
3.2 Реализация вычислительного ядра WebAssembly	23
3.2.1 Инициализация и загрузка модуля	23
3.2.2 Интерфейс обмена данными	24
3.3 Организация алгоритмических модулей	25
3.3.1 Унифицированный интерфейс модуля	25
3.3.2 Пример реализации: метод Гаусса	26
3.3.3 Ленивая загрузка модулей	26
3.4 Компонентная архитектура интерфейса	26
3.4.1 Компоненты-страницы	27
3.4.2 Компонент ввода данных MatrixInput	27
3.4.3 Компонент визуализации Visualization	27
3.4.4 Компонент журнала шагов StepLog	28

3.5	Утилитный модуль solverUtils	28
3.6	Маршрутизация и навигация	29
3.7	Система визуализации результатов регрессии	29
3.8	Темизация и стилизация	30
3.9	Конфигурация сборки	30
3.10	Итоги реализации	30
4	Эксперимент и анализ производительности	32
	Заключение	33
	Список литературы	34

Введение

В современной вычислительной науке численные методы занимают центральное место. Методы линейной алгебры, разложение матриц и итерационные алгоритмы оптимизации составляют фундамент, на котором строятся решения в инженерии, физике, экономике, науке о данных и искусственном интеллекте. Глубокое понимание этих методов необходимо для специалиста, работающего с реальными вычислительными задачами. Именно численные алгоритмы лежат в основе широкого класса прикладных проблем: от решения систем линейных уравнений высокой размерности и вычисления собственных значений матриц до спектрального анализа многомерных сигналов и численного интегрирования дифференциальных уравнений.

Однако имеется существенный разрыв между потребностями специалистов и доступными инструментами. Традиционные среды для численного анализа, такие как: MATLAB, Scilab, Mathematica, обладают широкими возможностями, но их использование сопряжено с рядом ограничений. Приобретение коммерческих лицензий требует значительных финансовых затрат, локальная установка и настройка среды создают дополнительный барьер для входа, а сами инструменты, как правило, ориентированы на опытного пользователя и не предоставляют средств для пошаговой интерактивной визуализации вычислительного процесса. Это делает их малопригодными как для первичного освоения алгоритмов, так и для быстрого исследовательского прототипирования в условиях ограниченных ресурсов.

Существующие веб-ориентированные альтернативы, например онлайн - калькуляторы, вычислительные блокноты или образовательные платформы, лишь частично закрывают эту потребность. Вычислительные среды требуют знания языков программирования и настройки серверной инфраструктуры, онлайн - калькуляторы не обеспечивают работу с данными нетривиальной размерности, а большинство образовательных ресурсов ограничиваются статическими иллюстрациями и не позволяют пользователю взаимодействовать с алгоритмом напрямую.

Технология WebAssembly (Wasm) открывает принципиально новые возможности для заполнения этой ниши. Стандартизированный бинарный формат, поддерживаемый всеми современными браузерами, позволяет компилировать высокопроизводительные библиотеки численных вычислений в код, исполняемый непосредственно на стороне клиента с производительностью, сопоставимой с нативной. Это принципиально отличает WebAssembly от предшествующих попыток перенести вычисления в браузер. В сочетании с современными средствами веб-визуализации - WebGL, D3.js, Canvas API - данная технология создаёт техническую основу для реализации платформы нового класса.

Настоящая дипломная работа посвящена проектированию и разработке веб-платформы интерактивной визуализации и анализа многомерных численных методов. Платформа предоставляет пользователю единую браузерную среду, в которой становится возможным не только выполнять вычисления над многомерными данными с использованием проверенных численных библиотек, но и наблюдать за поведением алгоритмов в интерактивном режиме: визуализировать промежуточные состояния, исследовать зависимость результата от входных параметров, сравнивать методы по точности и вычислительной сложности. Подобный подход позволяет совместить доступность браузерного решения с вычислительной мощностью нативных библиотек и наглядностью, необходимой для глубокого понимания алгоритмической сути рассматриваемых методов.

Постановка задачи

Целью данной работы является создание прототипа платформы для интерактивного анализа и визуализации многомерных численных методов – полезного инструмента для образования, который сделает сложные алгоритмы наглядными и доступными для изучения.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Провести анализ существующих инструментов и платформ для численного анализа и выработать на его основе требования к разрабатываемой платформе.
2. Исследовать технологии, применимые для реализации высокопроизводительных вычислений в браузере и обосновать выбор технического стека.
3. Спроектировать архитектуру и реализовать прототип платформы с поддержкой интерактивной визуализации выбранных численных методов и возможностью работы с многомерными входными данными.
4. Разработать методику оценки производительности платформы и провести вычислительный эксперимент в разных браузерах для сравнения эффективности созданной платформы.
5. Проанализировать полученные результаты и сформулировать рекомендации для дальнейшей доработки.

Обзор литературы

Формирование современных подходов к созданию высокопроизводительных вычислительных веб-приложений опирается на ряд исследований, посвящённых проектированию и стандартизации технологии WebAssembly, анализу её производительности, разработке линейно-алгебраических библиотек для веба, а также реализации интерактивной научной визуализации на стороне клиента. Анализ существующих публикаций позволяет обосновать выбор технологий и архитектурных решений, принятых в настоящей работе, и определить место разрабатываемой платформы среди существующих решений.

Фундаментальной работой, определившей развитие технологии WebAssembly, является статья [1], опубликованная на конференции PLDI 2017 и подготовленная инженерами из Google, Mozilla, Microsoft и Apple. В работе показано, что JavaScript, несмотря на оптимизации JIT-компиляторов, демонстрирует непредсказуемую производительность и не является надёжной целевой платформой для компиляции. WebAssembly решает эту проблему, предоставляя компактный бинарный формат с детерминированным поведением и скоростью выполнения, близкой к нативной.

Особый интерес представляют исследования, в который были проведены эксперименты по реализации матричных вычислений в WebAssembly и их сравнению с JavaScript-реализациями. Например, в отчете [2] реализовали библиотеку для работы с векторами и матрицами на Rust, скомпилированную в WASM через wasm-pack, и показали, что на операции умножения матриц размером до 1000×1000 WASM-реализация превосходит чистый JavaScript до 10 раз в Chrome и до 50 раз в Firefox. Кроме того, в работе [3] провели более детальный анализ на семи конфигурациях реализации умножения матриц (от 256×256 до 1024×1024), включая WebAssembly с SIMD-инструкциями и гибридные конфигурации с WebGPU, и выявили, что базовый WebAssembly без оптимизаций может быть даже медленнее JavaScript (ускорение 0.41x), поскольку JIT-компиляторы выполняют агрессивную оптимизацию паттернов доступа к памяти, - только комбинация SIMD-векторизации и блочного доступа к памяти позволяет WebAssembly превзойти JavaScript на 64

Ряд публикаций подтверждает техническую жизнеспособность компиляции существующих C/C++ библиотек в WebAssembly с использованием Emscripten для реализации интерактивной научной визуализации непосредственно в браузере. Так, в работе [4] скомпилировали libtiff и LAStools в WASM и продемонстрировали интерактивную визуализацию изоповерхности набора данных объёмом около 1 ТБ полностью в браузере с применением WebGPU: поверхность из 137,5 миллионов треугольников вычисляется за 526 мс и рендерится со скоростью 30 FPS. А в [5] описан подход к кроссплатформенной визуализации научных симуляций через WebAssembly

и WebGL без внешних зависимостей, где существующий C-код компилируется через Emscripten с поддержкой трёх режимов работы: нативного (OpenGL), гибридного (сервер + браузер) и полностью браузерного.

Опыт создания полноценных образовательных вычислительных сред в браузере представлен в статье [6]. Авторы реализовали браузерную среду разработки на Python, использующую Pyodide - порт CPython, скомпилированный в WebAssembly. Производительность системы составляет 3–5х от нативного CPython, что является приемлемым для образовательных задач. Практическое внедрение в школах Швейцарии подтверждает, что образовательные платформы на основе WebAssembly могут быть не только технически реализуемы, но и востребованы в учебном процессе.

Актуальность настоящего исследования подтверждается тем, что большая часть существующих работ либо фокусируется на отдельных аспектах, например, производительности или компиляции, либо ограничена специфическими кейсами без образовательной или визуализационной составляющей. Между тем, систематического сопоставления различных подходов в контексте создания комплексной вычислительной платформы с интерактивной визуализацией и образовательной направленностью в литературе крайне мало.

Рассмотренная литература подчёркивает как высокую значимость темы высокопроизводительных вычислений в браузере на основе WebAssembly, так и ограниченность имеющихся работ, объединяющих численные методы, интерактивную визуализацию и образовательную функциональность в комплексной, системной форме.

1 Технология WebAssembly и высокопроизводительные вычисления в браузерной среде

1.1. Определение и назначение WebAssembly

WebAssembly (Wasm) представляет собой открытый стандарт, определяющий переносимый бинарный формат инструкций для стековой виртуальной машины. Стандарт разработан рабочей группой W3C WebAssembly Working Group при участии инженеров компаний Google, Mozilla, Microsoft и Apple и опубликован в качестве рекомендации W3C в декабре 2019 года [1]. WebAssembly проектировался как целевая платформа компиляции для языков с явным управлением памятью, таких как C, C++ и Rust, и предназначен для выполнения вычислительно интенсивных задач в браузерной среде с производительностью, приближающейся к нативной.

Необходимость появления WebAssembly связана с эволюцией требований к веб-приложениям, которые всё чаще включают ресурсоёмкие вычисления: обработку графики, моделирование, анализ данных и выполнение алгоритмов, характерных для настольного программного обеспечения. Хотя современные JavaScript-движки, такие как V8, SpiderMonkey и JavaScriptCore, используют сложные механизмы оптимизации, включая JIT-компиляцию, специфика языка — динамическая типизация и автоматическое управление памятью — накладывает ограничения на достижимую эффективность и предсказуемость выполнения. В задачах, требующих строгого контроля над ресурсами и стабильной производительности, такие особенности могут становиться критическими.

WebAssembly предлагает альтернативную модель выполнения, ориентированную на эффективную доставку и исполнение кода в браузере. В отличие от JavaScript, распространяемого в текстовом виде и требующего многоступенчатой обработки, WebAssembly использует компактное бинарное представление, оптимизированное для быстрой загрузки и декодирования. Модуль WebAssembly компилируется в машинный код практически напрямую, минуя ряд промежуточных этапов, что сокращает время инициализации и снижает вариативность производительности во время выполнения. Такая архитектура делает WebAssembly особенно подходящим для реализации вычислительно интенсивных компонентов веб-приложений, приближая их характеристики к нативным.

1.2. Архитектура и модель исполнения

WebAssembly реализует стековую виртуальную машину с линейной памятью. Модуль содержит функции, таблицы, глобальные переменные и память; инструкции работают через стек операндов. Типовая корректность проверяется до выполнения, что исключает ошибки ти-

пов во время исполнения.

Поддерживаются четыре числовых типа: i32, i64, f32 и f64. Для вычислений используется f64, обеспечивающий высокую точность и детерминированную семантику операций, что гарантирует воспроизводимость результатов.

Линейная память представляет собой непрерывный расширяемый массив байтов с доступом через инструкции загрузки и сохранения, что соответствует модели C/C++. Память изолирована от хост-среды; взаимодействие с JavaScript осуществляется через типизированные массивы.

Модуль имеет секционную структуру (типы, функции, код, импорт, экспорт), что позволяет выполнять потоковую компиляцию. Исполнение происходит в изолированной песочнице: доступ к ресурсам среды возможен только через импортируемые функции, а выход за границы памяти приводит к исключению.

1.3. Компиляция нативного кода в WebAssembly

WebAssembly как целевая платформа компиляции поддерживается широким набором инструментов, различающихся по исходным языкам, архитектуре компилятора и характеристикам генерируемого кода. Рассмотрим основные подходы.

Emscripten представляет собой наиболее зрелый инструмент для компиляции кода на языках C и C++ в WebAssembly. Архитектурно Emscripten построен поверх компиляторной инфраструктуры LLVM: исходный код транслируется фронтендом Clang в промежуточное представление LLVM IR, после чего бэкенд генерирует бинарный модуль WebAssembly [4]. Помимо бинарного модуля, Emscripten генерирует JavaScript-обвязку (glue code), обеспечивающую загрузку модуля, инициализацию линейной памяти, эмуляцию стандартной библиотеки C и маршалинг данных между JavaScript и WebAssembly. Emscripten является не просто компилятором, а средой портирования: он позволяет перенести в браузер существующие кодовые базы на C/C++ (научные библиотеки, кодеки, игровые движки) с минимальными модификациями исходного кода. В выходной пайплайн Emscripten также интегрирован оптимизатор Binaryen [4].

Прямая компиляция через LLVM. Начиная с версии LLVM 8, бэкенд WebAssembly стабилизирован, что позволяет использовать компилятор Clang напрямую с целевой архитектурой wasm32 без привлечения Emscripten. Этот подход генерирует минимальные модули WebAssembly без JavaScript-обвязки, эмуляции файловой системы и прочего окружения. Разработчик самостоятельно обеспечивает связывание с библиотекой C (например, wasi-libc), настройку компоновщика wasm-ld и реализацию интерфейса с хост-средой. Подход пригоден для создания компактных автономных модулей, где требуется полный контроль над экспортируемым интерфейсом и размером бинарного файла [4].

WASI SDK развивает идею прямой компиляции через LLVM, объединяя Clang, компоновщик wasm-ld и реализацию стандартной библиотеки C (wasi-libc) в единый комплект. WASI (WebAssembly System Interface) определяет стандартизированный набор POSIX-подобных си-

стемных вызовов (файловый ввод-вывод, таймеры, генерация случайных чисел), которые модуль WebAssembly импортирует из хост-среды. Это отвязывает модуль от конкретного хоста: один и тот же бинарный файл может исполняться в браузере, на сервере (Wasmtime, Wasmer) или на периферийных устройствах. WASI SDK является основой направления «WebAssembly за пределами браузера» [11].

Rust и экосистема wasm-pack. Компилятор Rust поддерживает WebAssembly как целевую платформу первого класса через бэкэнд LLVM. Инструмент wasm-bindgen генерирует интерфейсный слой между Rust и JavaScript, обеспечивая типобезопасный маршалинг строк, структур и замыканий. Надстройка wasm-pack оркестрирует весь процесс сборки и упаковки результата в формат npm-пакета. Отсутствие сборщика мусора и модель владения памятью в Rust позволяют генерировать компактные модули WebAssembly без вспомогательного рантайма, что делает Rust одним из наиболее подходящих языков для высокопроизводительных Wasm-модулей [12]. AssemblyScript компилирует строго типизированное подмножество TypeScript непосредственно в WebAssembly. В отличие от перечисленных выше инструментов, AssemblyScript не использует LLVM: его компилятор транслирует абстрактное синтаксическое дерево в промежуточное представление Binaryen IR, после чего проходы оптимизации Binaryen генерируют итоговый модуль. Типовая система AssemblyScript принудительно использует Wasm-совместимые типы (i32, i64, f32, f64), а управление памятью объектов обеспечивается встроенным сборщиком мусора. AssemblyScript не является компилятором TypeScript или JavaScript в общем смысле: динамическая типизация, объединения типов и стандартная библиотека JS в нём не поддерживаются [13].

Компиляция языков с управляемой средой исполнения. Языки с автоматическим управлением памятью (Go, C#, Kotlin) используют два принципиально различных подхода к генерации WebAssembly. Первый подход состоит в компиляции виртуальной машины или интерпретатора в WebAssembly с последующим исполнением байт-кода внутри него: так устроен Blazor WebAssembly от Microsoft, исполняющий .NET IL через скомпилированный в Wasm рантайм Mono. Второй подход основан на предварительной (ahead-of-time) компиляции непосредственно в инструкции WebAssembly. Компилятор TinyGo, построенный на LLVM, транслирует подмножество Go в Wasm-модули, размер которых на порядок меньше выхода стандартного компилятора Go, за счёт упрощённого сборщика мусора и планировщика. Kotlin/Wasm использует третий путь: он опирается на предложение Wasm GC (управление памятью средствами хост-среды), перекладывая ответственность за сборку мусора на виртуальную машину браузера [14].

Binaryen занимает особое место в экосистеме WebAssembly как инструментарий постобработки, а не компилятор исходного кода. Его основная утилита wasm-opt принимает готовый модуль WebAssembly и применяет набор проходов оптимизации: удаление мёртвого кода, подстановка констант, упрощение потока управления, дедупликация. Binaryen используется как компонент внутри Emscripten, AssemblyScript, wasm-pack и других инструментариев. Применение wasm-opt позволяет сократить размер бинарного модуля на 10-30

Архитектурно все перечисленные инструментарии различаются по ключевому пара-

метру: объёму рантайма, включаемого в выходной модуль. Языки без сборщика мусора (C, C++, Rust, Zig) генерируют компактные модули без вспомогательного рантайма. Языки с управляемой памятью вынуждены либо включать рантайм в модуль (Go, .NET), либо опираться на ещё не повсеместно поддерживаемые расширения спецификации (Wasm GC). Этот компромисс между размером модуля, скоростью запуска и богатством исходного языка определяет выбор инструментария для конкретной задачи.

Передача числовых данных между JavaScript и WebAssembly, независимо от выбранного инструментария компиляции, требует явного управления линейной памятью модуля. Для передачи массива данных необходимо выделить область в линейном буфере, записать элементы через типизированное представление (`Float64Array` для чисел двойной точности), передать указатель в вызываемую функцию, считать результат и освободить память. Копирование выполняется ровно дважды (запись и чтение), а само вычисление протекает без дополнительных накладных расходов на маршалинг.

1.4. Многопоточность в браузерной среде

Браузерная среда исторически основана на однопоточной модели исполнения: основной поток обрабатывает пользовательские события, выполняет JavaScript-код и управляет отрисовкой интерфейса. Блокировка основного потока длительными вычислениями приводит к потере отзывчивости интерфейса. Механизм Web Workers позволяет создавать фоновые потоки исполнения, каждый из которых работает в изолированном контексте и обменивается данными с основным потоком через передачу сообщений [5].

Стандартный обмен данными между потоками основан на копировании, что создаёт существенные накладные расходы для больших числовых массивов. Объект `SharedArrayBuffer` решает эту проблему, предоставляя разделяемую область памяти, доступную из нескольких потоков одновременно. Синхронизация доступа обеспечивается атомарными операциями через объект `Atomics`. Emscripten транслирует вызовы POSIX `threads` в Web Workers, размещая линейную память WebAssembly-модуля в `SharedArrayBuffer`, что позволяет нескольким потокам работать с общими данными без копирования [5].

Использование `SharedArrayBuffer` было ограничено в 2018 году после обнаружения уязвимостей класса Spectre. Для восстановления поддержки W3C разработал механизм кросс-доменной изоляции (cross-origin isolation), требующий установки HTTP-заголовков `Cross-Origin-Embedder-Policy` (COEP) и `Cross-Origin-Opener-Policy` (COOP). Совместное применение этих заголовков создаёт изолированное окружение, в котором использование разделяемой памяти и высокоточных таймеров считается безопасным [6].

1.5. Производительность WebAssembly

Вопрос производительности WebAssembly по отношению к JavaScript и нативному коду является предметом активных исследований. В работе [7] продемонстрировано, что ре-

ализация матричного умножения на WebAssembly превосходит чистый JavaScript до 10 раз в Chrome и до 50 раз в Firefox при размерах матриц до 1000×1000 . Однако результаты работы [8] показывают, что наивная компиляция без оптимизации паттернов доступа к памяти может оказаться даже медленнее JavaScript (замедление до 2.4x), поскольку современные JIT-компиляторы агрессивно оптимизируют характерные паттерны работы с типизированными массивами. Только комбинация SIMD-векторизации и блочного доступа к памяти позволяет WebAssembly стабильно превосходить JavaScript [8].

Эти результаты подчёркивают, что простое использование WebAssembly не гарантирует автоматического прироста производительности. Ключевую роль играет качество компилируемого кода: алгоритмические оптимизации, реализованные в зрелых библиотеках (блочные алгоритмы, учёт иерархии кэш-памяти), сохраняют свою эффективность при компиляции в WebAssembly. Перенос существующей оптимизированной библиотеки через Emscripten даёт существенное преимущество перед реализацией тех же алгоритмов на JavaScript с нуля.

1.6. Спецификации BLAS и LAPACK

BLAS (Basic Linear Algebra Subprograms) представляет собой спецификацию интерфейсов для базовых операций линейной алгебры, впервые опубликованную в 1979 году [9]. Спецификация организована в три уровня. Уровень 1 определяет операции вектор-вектор со сложностью $O(n)$: скалярное произведение, нормы, масштабирование, линейные комбинации. Уровень 2 определяет операции матрица-вектор со сложностью $O(n^2)$: умножение матрицы на вектор, решение треугольных систем. Уровень 3 определяет операции матрица-матрица со сложностью $O(n^3)$: матричное умножение, ранговые обновления. Операции третьего уровня обладают высоким соотношением вычислений к обращениям в память ($O(n)$ операций на элемент данных), что позволяет эффективно использовать кэш-память и достигать производительности, близкой к пиковой [9].

LAPACK (Linear Algebra PACKage) представляет собой библиотеку подпрограмм для решения задач линейной алгебры высокого уровня: факторизации матриц, решения систем линейных уравнений, вычисления собственных значений и сингулярного разложения [10]. Подпрограммы LAPACK реализованы через вызовы BLAS, что позволяет автоматически наследовать оптимизации конкретной реализации BLAS для целевой платформы. Соглашение об именовании подпрограмм LAPACK отражает тип данных и характер операции: первая буква указывает на тип данных (например, d для double precision), последующие буквы кодируют тип матрицы (ge для общей, sy для симметричной, po для положительно определённой) и выполняемую операцию (sv для решения системы, trf для факторизации) [10].

OpenBLAS является одной из наиболее распространённых открытых реализаций стандартов BLAS и LAPACK. Библиотека написана на языках C и Fortran, содержит оптимизированные вычислительные ядра для широкого спектра процессорных архитектур и поддерживает многопоточное исполнение. Компиляция OpenBLAS средствами Emscripten в модуль WebAssembly позволяет использовать промышленную реализацию линейной алгебры непо-

средственно в браузерной среде, сохраняя алгоритмическую эффективность блочных методов при отсутствии архитектурно-зависимых SIMD-оптимизаций.

2 Проектирование архитектуры платформы

2.1. Требования к платформе и функциональные возможности

При проектировании веб-платформы интерактивной визуализации численных методов основной задачей являлось построение такой архитектуры, которая одновременно обеспечивала бы высокую вычислительную эффективность, наглядность представления алгоритмов, модульность расширения и удобство использования в браузерной среде. В отличие от обычных вычислительных программ, в данном случае система должна не только выдавать итоговый численный результат, но и демонстрировать пользователю структуру вычислительного процесса.

Это требование определяет необходимость разделения платформы на несколько согласованных уровней. Пользовательский уровень отвечает за ввод данных, навигацию и визуальное представление результатов. Логический уровень управляет сценариями решения задач и координирует выполнение конкретных численных методов. Вычислительный уровень обеспечивает выполнение ресурсоёмких операций линейной алгебры. Такое разделение позволяет уменьшить связанность подсистем и делает архитектуру пригодной для дальнейшего развития.

Архитектура платформы должна удовлетворять следующим требованиям: поддержка нескольких классов численных задач, единый принцип взаимодействия с алгоритмами, возможность пошаговой визуализации, использование высокопроизводительного вычислительного ядра и минимизация стартовой загрузки клиентского приложения. В соответствии с этими требованиями была выбрана многоуровневая клиентская архитектура, сочетающая компонентный интерфейс, модульную организацию алгоритмов и вычислительный слой на базе WebAssembly.

2.2. Выбор технологического стека

Проектирование платформы интерактивной визуализации численных алгоритмов требует обоснованного выбора технологий, обеспечивающих выполнение ключевых требований: высокопроизводительные вычисления на стороне клиента, модульная архитектура интерфейса, быстрая начальная загрузка и возможность интерактивной визуализации промежуточных состояний алгоритмов. В данном разделе последовательно рассматриваются решения, принятые для каждого уровня технологического стека.

Основу вычислительного ядра платформы составляет технология WebAssembly (WASM), выбор которой аргументирован результатами анализа в Главе 1. В качестве базового инструментария компиляции выбран Emscripten SDK, что продиктовано возможностью прямой трансляции высокооптимизированных нативных библиотек линейной алгебры, в частности OpenBLAS и LAPACK, в бинарный формат WASM. Данный подход позволяет использовать промышленно апробированные алгоритмы факторизации матриц и итерационных методов без потери их

алгоритмической сложности и эффективности, достигая производительности, сопоставимой с нативными приложениями.

Для реализации пользовательского интерфейса и управления состоянием приложения выбран фреймворк React.js. Использование компонентно-ориентированного подхода и механизма Virtual DOM позволяет эффективно синхронизировать визуальное представление с результатами вычислений в реальном времени. Интеграция высокопроизводительного WASM-модуля в среду выполнения JavaScript осуществляется посредством асинхронного интерфейса загрузки, что предотвращает блокировку основного потока выполнения (Main Thread) и обеспечивает отзывчивость интерфейса при обработке массивов данных значительной размерности.

Визуализационный слой платформы базируется на использовании Canvas API и библиотеки D3.js. Данный выбор обоснован необходимостью отрисовки динамически изменяющихся многомерных структур, где традиционные методы манипуляции DOM-деревом демонстрируют избыточные вычислительные затраты. Применение SharedArrayBuffer для передачи данных между вычислительным ядром и графическим слоем минимизирует накладные расходы на копирование объектов в памяти, что критически важно для обеспечения интерактивности при визуализации итерационных процессов. Таким образом, сформированный технологический стек образует когерентную среду, способную решать задачи численного анализа в рамках веб-браузера с соблюдением требований к точности и быстродействию.

2.3. Архитектура прототипа

Архитектура программного комплекса построена как многоуровневая клиентская система, в которой вычислительная, прикладная и визуальная части разделены по своим функциям, но работают согласованно в рамках единого пользовательского сценария. Такой подход позволяет, с одной стороны, обеспечить высокую производительность вычислений, а с другой стороны, сохранить интерактивность и наглядность работы системы.

В составе программного комплекса можно выделить три основных уровня:

- frontend-уровень
- вычислительный слой на основе WebAssembly
- слой визуализации результатов вычислений

Frontend-уровень отвечает за взаимодействие пользователя с системой. На данном уровне реализуются пользовательский интерфейс, маршрутизация между разделами приложения, ввод исходных данных, выбор численного метода и управление процессом вычислений. Кроме того, именно frontend-уровень координирует обмен данными между пользовательским интерфейсом, алгоритмическими модулями и вычислительным ядром.

Архитектурно frontend-уровень организован как набор страниц-контейнеров и переиспользуемых компонентов. Страницы отвечают за сценарий работы с конкретным классом задач: решение систем линейных алгебраических уравнений, решение нелинейных систем и

задачи аппроксимации или регрессии. Компоненты обеспечивают ввод данных, визуализацию промежуточных состояний и отображение журнала шагов.

Управление состоянием на frontend-уровне строится вокруг нескольких типов данных:

- состояния пользовательского ввода
- состояния загрузки вычислительного ядра
- состояния выполняемого алгоритма
- состояния визуализации и отображаемых результатов

Центральную роль в обмене с вычислительным ядром играет общий контекст выполнения, который инкапсулирует признак готовности Wasm-модуля и предоставляет интерфейс для вызова вычислительных процедур. Благодаря этому прикладные алгоритмы не взаимодействуют напрямую с низкоуровневыми механизмами WebAssembly, а используют единый абстрактный интерфейс. Такое решение уменьшает связанность компонентов и упрощает расширение системы.

Вычислительный слой реализован на основе Wasm-ядра и предназначен для выполнения ресурсоемких операций линейной алгебры. Его включение в архитектуру программного комплекса обусловлено тем, что чистая реализация вычислений на JavaScript не всегда обеспечивает необходимую производительность при работе с матрицами, векторами и итерационными методами.

Взаимодействие frontend-уровня с Wasm-ядром организовано через специальный интерфейс обмена данными. Пользовательские данные, подготовленные на frontend-уровне, сначала преобразуются в компактное представление, пригодное для передачи в вычислительный модуль. Далее они помещаются в линейную память WebAssembly, после чего вызывается экспортированная функция вычислительного ядра. По завершении вычисления результирующие данные считываются обратно из линейной памяти и преобразуются в структуры, удобные для дальнейшей обработки на стороне JavaScript.

С архитектурной точки зрения этот процесс включает несколько этапов:

1. формирование вычислительной команды и входных параметров на стороне JavaScript
2. выделение области в линейной памяти Wasm-модуля
3. запись входных данных в память модуля
4. вызов вычислительной функции ядра
5. получение результата в виде строки или структурированного текстового представления
6. преобразование результата в объекты JavaScript
7. освобождение ранее выделенной памяти

Такой механизм позволяет frontend-уровню использовать Wasm-ядро как специализированный вычислительный сервис. При этом сам модуль WebAssembly остается изолированным от логики интерфейса и не зависит от деталей визуального представления данных.

Слой визуализации отвечает за представление промежуточных и итоговых результатов вычислений в понятной для пользователя форме. Его задачей является не только отображение ответа, но и демонстрация структуры вычислительного процесса. Это особенно важно для системы учебного и исследовательского назначения, где необходимо показывать этапы преобразования матриц, итерации метода, невязки, аппроксимирующие кривые и другие характеристики решения.

Архитектурно слой визуализации связан как с frontend-уровнем, так и с вычислительным слоем. После завершения вычислительного шага прикладной модуль формирует данные для отображения и передает их в визуальные компоненты. Для различных типов задач применяются разные формы представления:

- для методов решения СЛАУ используются таблицы, отражающие текущее состояние матрицы и вектора правой части
- для итерационных нелинейных методов отображаются последовательности приближений и нормы невязки
- для задач регрессии и аппроксимации строятся графики, диаграммы, поля коэффициентов и метрики качества модели

Передача результатов в слой визуализации осуществляется не напрямую из Wasm-ядра, а через прикладной уровень. Это позволяет отделить вычислительное представление данных от визуального. Иначе говоря, вычислительное ядро возвращает результат в форме, удобной для математической обработки, а прикладной слой преобразует его в формат, пригодный для отрисовки таблиц, графиков и анимаций.

Таким образом, архитектура программного комплекса представляет собой согласованную систему взаимодействия трех уровней: frontend-уровень управляет пользовательским сценарием и состоянием приложения, Wasm-ядро выполняет численные вычисления, а слой визуализации обеспечивает интерпретацию и графическое представление результата. Такое разделение ответственности делает систему расширяемой, удобной для сопровождения и пригодной для дальнейшего развития.

2.4. Проектирование структур данных и интерфейсов взаимодействия

Для корректной работы программного комплекса необходимо было определить, в каком виде представлять многомерные данные в памяти, каким образом передавать их в вычислительное ядро и как организовать интерфейсы, обеспечивающие визуализацию промежуточных

и итоговых результатов. Проектирование структур данных и интерфейсов взаимодействия является ключевым этапом, так как именно на этом уровне связываются пользовательский ввод, вычислительные процедуры и графическое отображение.

Представление многомерных данных

Основными типами данных, используемыми в системе, являются:

- векторы
- матрицы
- наборы точек для задач регрессии и аппроксимации
- последовательности итерационных состояний
- структурированные результаты вычислений

На frontend-уровне многомерные данные хранятся в естественном для JavaScript виде. Векторы представлены одномерными массивами чисел, а матрицы - массивами строк, каждая из которых также является массивом чисел. Такое представление удобно для ввода, отображения и пошагового изменения данных в процессе визуализации.

Однако для передачи в вычислительное ядро этого недостаточно. BLAS- и LAPACK-подпрограммы предполагают последовательное размещение элементов матрицы в памяти. Поэтому перед вызовом Wasm-ядра матрицы преобразуются из двумерного представления в линейный массив значений. Иными словами, матрица размера $n \times n$ разворачивается в одномерную последовательность длины n^2 , где элементы располагаются в заранее определенном порядке. Аналогично векторы передаются как непрерывные одномерные массивы.

Таким образом, в системе используются два взаимосвязанных представления многомерных данных:

1. логическое представление - для работы на уровне интерфейса и алгоритмической логики
2. линейное представление - для передачи в память Wasm-модуля

Такой подход позволяет сохранить удобство работы с данными на уровне JavaScript и одновременно обеспечить совместимость с низкоуровневым вычислительным слоем.

Формат обмена с вычислительным ядром

Для взаимодействия с Wasm-ядром в системе используется унифицированный интерфейс вызова вычислительных процедур. Входные данные преобразуются в командное представление, содержащее имя вызываемой операции и список параметров. После этого подготовленная команда записывается в линейную память WebAssembly и передается на исполнение.

С архитектурной точки зрения такой интерфейс удобен по нескольким причинам. Во-первых, он скрывает от прикладной логики детали работы с линейной памятью. Во-вторых,

он позволяет использовать единый механизм вызова для различных численных методов. В-третьих, он облегчает добавление новых операций без изменения общего принципа взаимодействия между JavaScript и Wasm.

Результат работы вычислительного ядра возвращается в структурированном текстовом формате, после чего на стороне JavaScript выполняется его разбор. В зависимости от типа задачи результат может быть интерпретирован как:

- скалярное значение
- вектор решения
- матрица коэффициентов
- метрика качества
- диагностическая информация о ходе вычисления

После разбора данные преобразуются в объекты прикладного уровня и передаются либо в алгоритмический модуль, либо непосредственно в слой визуализации.

Интерфейсы взаимодействия между уровнями

Для связи между слоями системы спроектированы интерфейсы нескольких типов.

Первый тип - интерфейсы пользовательского ввода. Они отвечают за получение размерности задачи, матриц, векторов, начальных приближений, параметров итерационного процесса и экспериментальных точек. Эти данные приводятся к внутреннему формату и передаются в алгоритмический модуль.

Второй тип - интерфейсы вычислительного взаимодействия. Они обеспечивают вызов Wasm-ядра и возврат результата в прикладной слой. Главная задача этого интерфейса заключается в том, чтобы скрыть от верхнего уровня детали работы с памятью, преобразованием типов и внутренним форматом вычислительного модуля.

Третий тип - интерфейсы визуализации. Они используются для отображения состояния алгоритма в процессе его выполнения. Такие интерфейсы позволяют:

- задавать текущий статус вычисления
- отображать промежуточные шаги
- обновлять отдельные элементы матриц и векторов
- передавать данные для построения графиков
- формировать итоговые блоки результата

Именно наличие выделенного интерфейса визуализации позволяет системе не ограничиваться выдачей конечного ответа, а показывать пользователю внутреннюю структуру вычислительного процесса.

Перехват промежуточных итераций

Одним из важнейших требований к системе является возможность отрисовки промежуточных состояний алгоритмов. Для этого в архитектуре предусмотрен механизм перехвата вычислительных шагов на уровне прикладной логики. Идея заключается в том, что алгоритм не выполняется как единый "черный ящик", а организуется как последовательность этапов, каждый из которых может быть зафиксирован и передан в слой визуализации.

На практике это означает, что после каждого значимого шага алгоритма формируется снимок текущего состояния. В такой снимок могут входить:

- текущее состояние матрицы
- текущее приближение вектора решения
- номер итерации
- значение невязки
- описание выполняемой операции
- результат очередного вычислительного вызова

Далее этот снимок передается через интерфейс визуализации в компонент отображения. За счет этого система может не только показать конечный результат, но и воспроизвести процесс его получения шаг за шагом. Для прямых методов это выражается в отображении преобразований строк и столбцов матрицы, для итерационных методов - в последовательности приближений, а для регрессионных задач - в построении графиков по итоговым и промежуточным данным.

Структуры данных для визуализации

Для нужд визуализации в системе проектируются специальные структуры результатов. Они отличаются от исходных вычислительных структур тем, что ориентированы не на математическую обработку, а на графическое отображение.

Для табличной визуализации используются структуры, содержащие:

- значения элементов матрицы
- координаты выделяемых строк и столбцов
- информацию о текущем шаге
- текстовые пояснения

Для итерационных методов используются структуры вида:

- номер итерации
- текущее приближение

- значение нормы ошибки или невязки
- дополнительная диагностическая информация

Для задач регрессии и аппроксимации используются структуры, описывающие:

- исходные точки наблюдений
- рассчитанные коэффициенты модели
- значения функции на наборе узлов
- остатки
- метрики качества
- данные для построения графиков и диаграмм

Благодаря такому проектированию вычислительный результат переводится в унифицированную форму, пригодную для дальнейшей отрисовки средствами интерфейса.

Итоги проектирования интерфейсов

Таким образом, при проектировании структур данных и интерфейсов взаимодействия была выбрана схема, в которой логическое представление данных отделено от их низкоуровневого представления в памяти Wasm, а вычислительные интерфейсы отделены от интерфейсов визуализации. Это решение обеспечивает несколько преимуществ:

- упрощает организацию обмена между JavaScript и WebAssembly
- позволяет хранить данные в удобной для прикладной логики форме
- делает возможной пошаговую фиксацию промежуточных состояний
- упрощает построение графиков, таблиц и анимаций
- упрощает построение графиков, таблиц и анимаций;
- облегчает расширение системы новыми алгоритмами и новыми типами визуализации

Следовательно, спроектированные структуры данных и интерфейсы взаимодействия образуют основу, на которой строится как вычислительная, так и визуальная часть программного комплекса.

3 Реализация прототипа

3.1. Файловая структура проекта

В данной главе описывается реализация прототипа веб-платформы интерактивной визуализации многомерных численных алгоритмов. Рассматриваются файловая структура проекта, реализация вычислительного ядра на базе WebAssembly, организация алгоритмических модулей, компонентная архитектура пользовательского интерфейса, система интерактивной визуализации, а также механизмы темизации и маршрутизации. Реализация выполнена в соответствии с архитектурными решениями, описанными в Главе 2.

Проект организован в соответствии с компонентным подходом, принятым в экосистеме React. Каждый функциональный модуль вынесен в отдельный каталог, что обеспечивает наглядное разделение ответственности и упрощает навигацию по кодовой базе. Корневой каталог содержит конфигурационные файлы сборщика Vite, манифест зависимостей `package.json` и директорию с бинарными артефактами WebAssembly.

```
src/
|-- App.jsx                # Корневой компонент, маршрутизация
|-- main.jsx              # Точка входа приложения
|-- pages/
|   |-- HomePage.jsx      # Главная страница с Canvas-анимацией
|   |-- AlgorithmsPage.jsx # Каталог алгоритмов с фильтрацией
|   |-- SolverPage.jsx    # Страница решателей СЛАУ
|   |-- NonlinearSolverPage.jsx # Страница нелинейных методов
|   `-- RegressionPage.jsx # Страница регрессии и аппроксимации
|-- components/
|   |-- ui/
|   |   |-- Header.jsx    # Навигационный заголовок
|   |   |-- Modal.jsx     # Модальное окно описания алгоритма
|   |   |-- ThemeToggle.jsx # Переключатель тёмной/светлой темы
|   |   `-- AlertModal.jsx # Модальное окно ошибок и уведомлений
|   `-- solver/
|       |-- MatrixInput.jsx # Ввод матрицы A и вектора b
|       |-- Visualization.jsx # Интерактивная визуализация
|       |-- StepLog.jsx     # Журнал пошагового решения
|       `-- WasmStatus.jsx  # Индикатор загрузки WASM-модуля
|-- hooks/solvers/
|   |-- linear/            # 10 методов решения СЛАУ
|   |-- nonlinear/         # 5 методов нелинейных систем
```

```

|    `-- approx/                                # 5 методов аппроксимации
|-- data/
|    `-- algorithms.js                          # Каталог алгоритмов и метаданные
|-- utils/
|    `-- solverUtils.js                        # Утилиты визуализации и BLAS-обёртки
|-- wasm/
|    `-- WasmContext.jsx                      # React Context для WASM-ядра
`-- styles/
    `-- app.css                              # Глобальные стили и CSS-переменные

blas_wasm/
|-- shell_cblas.js                            # Emscripten-загрузчик WASM-модуля
|-- shell_cblas.wasm                         # Скомпилированный модуль OpenBLAS
`-- shell_cblas.worker.js                   # Web Worker для многопоточности

```

Такая организация позволяет добавлять новые алгоритмы без изменения существующего кода: достаточно создать новый файл в соответствующем подкаталоге `hooks/solvers/`, зарегистрировать маршрут в `App.jsx` и добавить описание в `algorithms.js`.

3.2. Реализация вычислительного ядра WebAssembly

Вычислительное ядро платформы реализовано как модуль WebAssembly, скомпилированный из библиотеки OpenBLAS с помощью Emscripten SDK. Модуль предоставляет интерфейс для выполнения операций BLAS и LAPACK непосредственно в браузере, обеспечивая производительность, сопоставимую с нативными приложениями.

3.2.1. Инициализация и загрузка модуля

Загрузка WASM-модуля реализована через компонент-провайдер `WasmProvider`, построенный на основе React Context API. При монтировании провайдера в DOM динамически создаётся элемент `<script>`, загружающий файл `shell_cblas.js` — сгенерированный Emscripten загрузчик. Объект `window.Module` конфигурируется до загрузки скрипта, что позволяет задать пути к файлам, обработчики вывода и колбэк завершения инициализации:

```

window.Module = {
  locateFile: function(path) { return '/blas_wasm/' + path; },
  print: function(...args) { _printHandler(...args); },
  printErr: function() {},
  onRuntimeInitialized: function() {
    setWasmReady(true);
    window.Module._main(0, 0);
  }
};

```



```

    }
};

```

Флаг `loadingRef` предотвращает повторную загрузку при перемонтировании компонента в режиме Strict Mode. После успешной инициализации состояние `wasmReady` становится истинным, и все зависимые компоненты получают доступ к вычислительному ядру через хук `useWasm()`.

3.2.2. Интерфейс обмена данными

Для взаимодействия с WASM-ядром реализована функция `runBlas`, инкапсулирующая полный цикл обмена данными между JavaScript и WebAssembly. Входная команда представляет собой текстовую строку, содержащую имя BLAS- или LAPACK-подпрограммы и её параметры:

```

const runBlas = useCallback((cmd) => {
  let output = '';
  const prev = _printHandler;
  _printHandler = (t) => { output += t + '\n'; };

  const len = window.Module.lengthBytesUTF8(cmd) + 1;
  const ptr = window.Module._malloc(len);
  window.Module.stringToUTF8(cmd, ptr, len);
  window.Module._run_command(ptr);
  window.Module._free(ptr);

  _printHandler = prev;
  return output.trim();
}, []);

```

Механизм обмена включает следующие этапы: выделение памяти в линейном пространстве WASM-модуля через `_malloc`, запись UTF-8-строки команды в эту память через `stringToUTF8`, вызов экспортированной функции `_run_command`, перехват результата через подменяемый обработчик `_printHandler` и освобождение памяти через `_free`. Мутабельный обработчик `_printHandler` решает проблему захвата ссылки Emscripten на функцию `Module.print` при инициализации, поэтому перенаправление вывода реализовано через косвенный вызов.

Результат вычислений возвращается в виде текстовой строки, которая разбирается утилитными функциями `parseVec`, `parseMat` и `parseScalar` из модуля `solverUtils.js`. Такой подход позволяет использовать единый механизм для всех типов BLAS- и LAPACK-операций.

3.3. Организация алгоритмических модулей

Каждый численный метод реализован как самостоятельный модуль — JavaScript-файл, экспортирующий объект-конфигурацию. Модули распределены по трём категориям в соответствии с классификацией задач.

Таблица 3.1: Реализованные алгоритмические модули

Категория	Количество	Методы
Многомерные СЛАУ	10	Гаусс, LU, QR, Холецкого, Якоби, Зейделя, SOR, MinRes, BiCG, GMRES
Нелинейные системы	5	Ньютон, Бroyден, простые итерации, гомотопия, продолжение по параметру
Аппроксимация	5	Линейная регрессия, полиномиальная регрессия, RBF, наименьшие квадраты, сплайны

3.3.1. Унифицированный интерфейс модуля

Каждый модуль экспортирует объект фиксированной структуры, содержащий метаданные и асинхронную функцию решения. Ниже приведён обобщённый шаблон:

```
export default {  
  title: 'Название метода',  
  subtitle: 'Математическое описание',  
  prefix: 'url-slug',  
  defaultSize: 3,  
  exampleSize: 3,  
  exampleA: [...], [...], [...],  
  exampleB: [...],  
  stepDelay: 600,  
  
  async solve(ctx) {  
    const { data, runBlas, viz, stepLog, skipRef } = ctx;  
    // Реализация алгоритма с визуализацией  
  }  
};
```

Контекст выполнения `ctx` передаётся страницей-контейнером и содержит пользовательские данные `data`, функцию вызова BLAS-ядра `runBlas`, ссылки на компоненты визуализации `viz` и журнала шагов `stepLog`, а также флаг пропуска анимации `skipRef`. Такая декомпозиция позволяет алгоритмическим модулям работать независимо от деталей пользовательского интерфейса.

3.3.2. Пример реализации: метод Гаусса

Рассмотрим реализацию метода Гаусса как характерный пример построения алгоритмического модуля. Функция `solve` организована в два параллельных потока выполнения: анимированный и вычислительный.

Анимированный поток отвечает за пошаговое отображение преобразований матрицы. На каждом шаге прямого хода выполняются выбор ведущего элемента по столбцу с подсветкой кандидатов, перестановка строк при необходимости, элиминация с отображением формулы операции и покомпонентным обновлением ячеек. Между операциями вставляются управляемые паузы через `animSleep`.

Вычислительный поток формирует журнал BLAS-вызовов. На каждом шаге алгоритма генерируются текстовые команды для WASM-ядра: `cblas_idamax` для поиска максимального элемента, `cblas_dswap` для перестановки строк и `cblas_daxpy` для элиминации. Обратная подстановка выполняется единым вызовом `cblas_dtrsv`, а верификация — через LAPACK.

Механизм пропуска анимации реализован через разделяемую ссылку `skipRef`. При активации пользователем все паузы мгновенно разрешаются, а матрица пересчитывается без визуальных обновлений и записывается в финальном виде.

3.3.3. Ленивая загрузка модулей

Для минимизации начального объёма загрузки применена стратегия ленивого импорта. В корневом компоненте `App.jsx` каждый алгоритмический модуль зарегистрирован как фабричная функция динамического импорта:

```
const solverConfigs = {
  gauss: () => import('./hooks/solvers/linear/useGauss'),
  lu:    () => import('./hooks/solvers/linear/useLU'),
  qr:    () => import('./hooks/solvers/linear/useQR')
};
```

Загрузка модуля происходит только при переходе на соответствующий маршрут. Страницы-контейнеры также загружаются лениво через `React.lazy` с оборачиванием в `Suspense`. Такой подход позволяет Vite разбивать приложение на изолированные чанки, значительно сокращая время первоначальной загрузки.

3.4. Компонентная архитектура интерфейса

Пользовательский интерфейс построен на основе библиотеки React 19 с применением функциональных компонентов и хуков. Компоненты организованы в два уровня: компоненты-страницы и переиспользуемые UI-компоненты.

3.4.1. Компоненты-страницы

HomePage — главная страница платформы. Она содержит анимированную Canvas-сетку, реагирующую на движение мыши, счётчики с анимацией подсчёта и карточки навигации по категориям алгоритмов. Анимация реализована через `requestAnimationFrame` с физической моделью пружинного возврата узлов сетки.

AlgorithmsPage — каталог всех реализованных алгоритмов. Компонент поддерживает фильтрацию по категориям через параметр `location.state` и отображение модальных окон с описаниями алгоритмов. Метаданные загружаются из централизованного реестра `algorithms.js`.

SolverPage — универсальная страница для решения СЛАУ. Она объединяет компоненты ввода матрицы, визуализации и журнала шагов. Управление состоянием решения реализовано через `useRef` для императивного доступа к дочерним компонентам и `useState` для реактивного отслеживания процесса.

RegressionPage — страница для задач аппроксимации и регрессии. В отличие от `SolverPage` она реализует табличный ввод данных наблюдений с динамическим количеством признаков от 1 до 5 и строк, а также собственную систему визуализации результатов на основе Canvas API, включающую графики рассеяния, остатков, гистограммы, корреляционные матрицы и диаграммы коэффициентов.

3.4.2. Компонент ввода данных `MatrixInput`

Компонент `MatrixInput` реализует динамический ввод матрицы A и вектора b произвольной размерности от 1×1 до 11×11 . Используется паттерн `forwardRef` с `useImperativeHandle` для экспонирования императивного API родительскому компоненту.

- `buildGrid(n)` — программная генерация таблицы ввода заданной размерности;
- `readInput()` — считывание и валидация всех введённых значений;
- `loadExample(A, b, n)` — заполнение полей предустановленным примером.

DOM-элементы таблицы создаются императивно для обеспечения высокой производительности при частых обновлениях размерности. Каждая ячейка идентифицируется атрибутом `id` формата `{prefix}-a-{i}-{j}`, что позволяет осуществлять прямой доступ к элементам при загрузке примеров и валидации.

3.4.3. Компонент визуализации `Visualization`

Компонент `Visualization` является центральным элементом интерактивного представления алгоритма. Он предоставляет алгоритмическому модулю набор методов для управления отображением.

- `setStatus(text)` и `setOpLabel(text)` — отображение текущего шага и формулы операции;

- `setContainerHTML(html)` — отрисовка интерактивной таблицы матрицы;
- `enableBacksub(runner)` — активация кнопки обратной подстановки с передачей функции анимации;
- `getSpeed()` — получение текущей скорости анимации: 0.25x, 0.5x или 1x.

Компонент поддерживает двухуровневую визуализацию: основную для прямого хода алгоритма и вторичную для обратной подстановки, каждая со своим набором элементов управления. Вторичная секция монтируется по требованию и запускает анимацию через `useEffect` после завершения монтирования.

3.4.4. Компонент журнала шагов `StepLog`

Компонент `StepLog` отвечает за формирование протокола вычислений, воспроизводящего последовательность BLAS-вызовов. Каждый шаг содержит заголовок, текстовое описание, HTML-представление матрицы на данном этапе и BLAS-команду. Шаги добавляются через метод `addStep` в скрытом состоянии и раскрываются с задержкой через `showStep`, создавая эффект последовательного появления. Скроллинг к последнему шагу обеспечивает навигацию в длинных протоколах.

3.5. Утилитный модуль `solverUtils`

Модуль `solverUtils.js` содержит разделяемые утилиты, используемые всеми алгоритмическими модулями. Функции модуля можно разделить на четыре группы.

Функции форматирования и разбора обеспечивают преобразование данных между JavaScript и текстовым форматом BLAS-ядра: `fmtNum` форматирует числа с подавлением незначащих нулей, `parseVec` и `parseMat` извлекают векторы и матрицы из текстового вывода WASM-модуля, а `parseScalar` разбирает скалярный результат.

Функции визуализации матриц генерируют HTML-таблицы для различных представлений: `renderAugmented` строит расширенную матрицу $[A|b]$ с выделением ведущего элемента и строк элиминации, `renderMatrix` отображает отдельную матрицу с подсветкой частей L и U , а `renderImat` формирует интерактивную таблицу с атрибутами `data-row` и `data-col` для покомпонентного обновления.

Функции анимации управляют интерактивным отображением: `updateCell` обновляет значение ячейки с CSS-анимацией перехода, `highlightRow` и `highlightCell` подсвечивают элементы цветовыми классами, `clearHighlights` сбрасывает все выделения, а `animSleep` реализует управляемую паузу, мгновенно разрешаемую при активации пропуска.

Математические функции и функции верификации включают `matVec`, `matTVec`, `dot` и `norm` для промежуточных вычислений на JavaScript, `solutionHtml` для формирования блока результата и `verifyWithDgesv` для независимой проверки решения через LAPACK.

3.6. Маршрутизация и навигация

Маршрутизация реализована на основе библиотеки React Router DOM v7. Корневой компонент `App.jsx` определяет маршруты для главной страницы, каталога алгоритмов и каждого алгоритмического модуля. Каждый маршрут привязан к компоненту-обёртке, который передаёт соответствующий загрузчик конфигурации в страницу-контейнер:

```
<Route path="/gauss" element={<SolverRoute solverKey="gauss" />} />
<Route path="/newton" element={<NonlinearRoute solverKey="newton" />} />
<Route path="/linear-regression" element={<RegressionRoute solverKey="li
regression" />} />
```

Навигация между страницами осуществляется программно через хук `useNavigate`. Переход с главной страницы в каталог выполняется с передачей фильтра категории через объект `state`, что позволяет странице алгоритмов сразу отображать только выбранный класс методов.

Компонент `Header` присутствует на всех страницах и содержит логотип-ссылку на главную, кнопки модальных окон «О нас» и «Контакты», а также переключатель темы. Страницы решателей дополнительно содержат кнопку возврата в каталог алгоритмов.

3.7. Система визуализации результатов регрессии

Для задач аппроксимации и регрессии реализована система построения графиков на основе Canvas API без использования сторонних библиотек. Компонент `RegressionPage` содержит встроенный модуль рендеринга, поддерживающий шесть типов визуализаций.

1. карточка уравнения модели, отображающая формулу регрессии и коэффициент детерминации R^2 с цветовой индикацией качества;
2. метрики качества, представленные набором карточек с числовыми показателями модели, включая MSE, MAE и RMSE;
3. график рассеяния с регрессионной кривой, включающий точки наблюдений, аппроксимирующую кривую и линии невязок;
4. графики частичной зависимости для многомерных данных, демонстрирующие влияние каждого признака на отклик при фиксации остальных;
5. диаграмма «Предсказанное vs Фактическое», отображающая отклонение точек от диагонали идеального предсказания;
6. визуализация остатков и гистограмма их распределения.

Все графики адаптированы к тёмной и светлой темам: цветовые палитры определяются динамически на основе текущего значения атрибута `data-theme`. Для обеспечения чёткости на экранах с высокой плотностью пикселей применяется масштабирование Canvas через `devicePixelRatio`.

3.8. Темизация и стилизация

Система стилей построена на CSS-переменных, определённых в корневом элементе `:root`. Платформа поддерживает две темы — светлую по умолчанию и тёмную, переключаемую атрибутом `data-theme="dark"` на элементе `<html>`.

Для каждой темы определён полный набор переменных, охватывающих фоны, цвета текста, акцентные цвета категорий, свечения, границы и тени. Переключение темы не вызывает перерисовку React-дерева: применяются только CSS-правила. Для Canvas-визуализаций цвета темы считываются через `getComputedStyle` с отслеживанием изменений через `MutationObserver`.

Цветовая схема категорий алгоритмов обеспечивает визуальное различие: синий для СЛАУ, индиго для нелинейных систем и янтарный для задач аппроксимации. Эти цвета используются в карточках каталога, индикаторах и модальных окнах.

3.9. Конфигурация сборки

Сборка проекта выполняется с помощью Vite 8 — инструментария, обеспечивающего мгновенную горячую перезагрузку в режиме разработки и оптимизированную production-сборку на базе Rollup.

Для корректной работы WebAssembly в браузере конфигурация Vite задаёт HTTP-заголовки изоляции: `Cross-Origin-Opener-Policy` со значением `same-origin`, `Cross-Origin-Embedder-Policy` со значением `require-corp` и `Cross-Origin-Resource-Policy` со значением `same-origin`. Эти заголовки необходимы для активации `SharedArrayBuffer` требуемого для многопоточных вычислений в WASM.

Для обслуживания WASM-артефактов реализован кастомный плагин `Vite blasWasmPlugin`. В режиме разработки плагин перехватывает запросы к `/blas_wasm/*` и транслирует их в чтение файлов из директории `blas_wasm/` с корректными MIME-типами и заголовками CORS. В режиме сборки плагин эмитирует артефакты `shell_cblas.js`, `shell_cblas.wasm` и `shell_cblas.worker.js` в каталог `dist/blas_wasm/` через метод `this.emitFile`. Такое решение позволяет избежать дублирования файлов в `public/` и обеспечивает единый источник WASM-бинарников.

Стек зависимостей проекта минимален: React 19.2, React Router DOM 7.13 и Vite 8 с плагином `@vitejs/plugin-react`. Отсутствие тяжёлых библиотек визуализации обусловлено решением реализовать графики средствами Canvas API, что снижает объём итогового бандла и устраняет зависимость от сторонних обновлений.

3.10. Итоги реализации

В результате реализации создан прототип веб-платформы, включающий 20 алгоритмических модулей, объединённых унифицированным интерфейсом взаимодействия с WASM-ядром. Основные характеристики реализации состоят в следующем.

- модульная архитектура с ленивой загрузкой, обеспечивающая расширяемость и минимальное время начальной загрузки;
- полноценная интеграция библиотеки OpenBLAS через WebAssembly с текстовым протоколом обмена данными;
- двухуровневая система визуализации: интерактивная анимация алгоритма в реальном времени и детальный протокол BLAS-вызовов;
- адаптивная система стилей с поддержкой тёмной и светлой тем без перемонтирования компонентов;
- система визуализации результатов регрессии на базе Canvas API с шестью типами графиков;
- кастомный плагин Vite для обслуживания WASM-артефактов без дублирования файлов.

Все архитектурные решения, спроектированные в Главе 2, были реализованы в полном объёме. Трёхуровневая структура, включающая frontend, вычислительное ядро и визуализацию, обеспечила разделение ответственности, а унифицированный интерфейс алгоритмических модулей позволил стандартизировать добавление новых методов.

4 Эксперимент и анализ производительности

Заключение

Список литературы

- [1] Joseph Y-T. Leung Handbook of Scheduling: Algorithms, Models, and Performance Analysis CRC Press 2004
- [2] Michelle Galin, Anna Hedblom Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads Linköping University 2025
- [3] Jinsong Mao, Benjamin E. Ujcich, Shiqing Ma Rethinking Provenance Completeness with a Learning-Based Linux Scheduler arXiv 2025
- [4] Xinbo Wang, Shian Jia, Ziyang Huang, Jing Cao, Mingli Song Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies arXiv 2025
- [5] Ion Stoica, Hussein Abdel-Wahab Earliest Eligible Virtual Deadline First : A Flexible and Accurate Mechanism for Proportional Share Resource Allocation Old Dominion University 1996
- [6] A1349 Pavel Shago [Электронный ресурс]. – Режим доступа: <https://github.com/shgpavel/A1349> – Дата обращения 16.12.2025.
- [7] Andrew S. Tanenbaum, Herbert Bos MODERN OPERATING SYSTEMS Pearson Education 2023
- [8] Liz Rice Learning eBPF O'Reilly Media, Inc 2023
- [9] Torvalds L. Linux Kernel [Электронный ресурс] / L. Torvalds. – Режим доступа: <https://github.com/torvalds/linux> – Дата обращения 04.12.2025.
- [10] sched-ext/scx [Электронный ресурс]. – Режим доступа: <https://github.com/sched-ext/scx> – Дата обращения 07.12.2025.
- [11] lkml [Электронный ресурс]. – Режим доступа: <https://lkml.org> – Дата обращения 24.11.2025.
- [12] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Rombakh, Paul Turner, Christoph Lameter ghOST: Scheduling Beyond the OS ACM SIGOPS 2021